interacting with capacity warnings.
This includes service owners and the Scalability group doing the triage, and we ask everyone who is using capacity warnings in some form to give feedback.

The key question to answer when presented a capacity warning is: Is this useful for me and/or my wider context?

We acknowledge that whether a capacity warning is seen as useful or not useful is highly dependent on the individual working with these warnings and their respective context.
Hence we collect feedback from everyone individually, so we ideally have multiple datapoints per capacity warning.

Feedback can be given at any point in time but ideally reflects back on when the capacity warning got created.
Did it turn out meaningful after all?

The process looks like this:

1. When a capacity warning is generated, Tamland creates an issue in the respective capacity warning tracker.
2. Team member feedback can be given through a comment on the issue
3. The comment includes either the ~tamland-feedback:useful or ~tamland-feedback:not-useful label to signal whether or not the capacity warning was useful to the team member.
   1. Required: When using the ~tamland-feedback:not-useful label, we ask to also include a brief explanation why this wasn't useful for you specifically (in the same comment).
   1. Optional: Feel free to leave additional feedback for a useful capacity warning, too.

The team member running triage is asked to ideally leave a feedback for each open and newly created capacity warning.

For example, a forecast can be considered useful in these cases:

1. The forecast indicates impending saturation, which is highly relevant to prevent an incident.
2. A forecast looked "odd enough" to strike up a conversation about capacity limitations, which helped prioritize work. In this case, the forecast can be technically inaccurate, but still useful.

Cases for labeling a capacity warning as not-useful include:

1. The underlying timeseries is so jumpy that the projection is obviously not meaningful.
2. A recent trend has been picked up too early and the projection is obviously overly pessimistic with no action required for anyone to mitigate or research further.

Key Performance Indicator: Precision

Based on the feedback data collected in comments, we derive a key performance indicator Precision and define this as follows.

1. A capacity warning is considered useful, if it has at least one useful vote.

2. For a set of rated issues i, we calculate precision as the ratio of useful issues in i to total issues in i.
3. When referring to precision as a KPI over time, we use the creation timestamp for a capacity warning.

In addition to precision, we also define a KPI rated to indicate the ratio of rated issues to total issues for a given time period.

GitLab Dedicated Capacity Planning

The following details the team-level agreements and responsibilities regarding capacity planning for GitLab Dedicated.
While capacity planning for GitLab.com is a shared activity, capacity planning for GitLab Dedicated implements a more differentiated responsibility model.

Stakeholders: Observability team and Dedicated teams

1. The Dedicated team is responsible for defining saturation metrics Tamland monitors, and to configure tenants for capacity planning.
1. The Dedicated team runs Tamland inside tenant environments and produces saturation forecasting data.
1. The Observability team owns the reporting side of capacity planning and makes sure reports and warnings are available.
1. The Dedicated team is responsible for triaging and responding to the forecasts and warnings generated, and applying any insights to Dedicated tenant environments.
1. The Observability team implements new features and fixes for Tamland to aid the capacity planning process for GitLab Dedicated.

Defining saturation metrics and tenants

Saturation points and services can be set up for Tamland monitoring through the Tamland manifest.
The manifest is generated from the GET metrics catalog using a jsonnet generator.

Executing Tamland

Tamland runs inside tenant environments on a daily cadence and produces forecasting data to a S3 bucket.
For more information, please refer to documentation and this project.

Reporting and capacity warnings

The operational project to implement capacity planning for GitLab Dedicated is gitlab-dedicated.
This project runs a scheduled pipeline which produces the Tamland report and manages capacity warnings for configured tenant environments.

Triage and Response

Tamland manages capacity warnings in the gitlab-dedicated issue tracker.

In terms of labels used, the same mechanics apply as for GitLab.com (see above).

Strategy for Increasing Capacity

The strategy for handling potential capacity planning issues for GitLab Dedicated is different from GitLab.com in several ways:

1. For GitLab Dedicated, we strive for homogeneity of the tenant environments, particularly for tenants using the same reference architecture. This needs to be considered when deciding on a course of action for a potential capacity issue.
1. Changes that increase capacity should consider whether to be applied at the tenant level, to the reference architecture, as an overlay on a reference architecture, or globally.
1. Additional per-tenant costs should be considered as part of the triage and response process and increases should be approved by the Dedicated Product Manager (see this issue template).
1. Depending on where the capacity is increased (on the local to global spectrum), the change should also be considered from a cost-vs-complexity trade-off. Different situations may require different trade-offs.

Dedicated Capacity Planning is in it's early stage, and we can expect that the process will be iterated on rapidly as our experience in this area increases.

Feature development

We strive to provide reporting and capacity warnings tailored to our needs.
In particular for GitLab Dedicated, we anticipate the need to provide more tailored reporting and capacity warnings in terms of presentation and workflow.
This work will be managed through the Scalability issue tracker.

More general Tamland development is managed through Tamland's issue tracker.

Examples of Capacity Issues

In this section, we discuss a few capacity planning issues and describe how we applied the process above when addressing them.

redis-cache / redisprimarycpu potential saturation

gitlab-com/gl-infra/capacity-planning364

We split out some operations from redis-cache to a new redis-repository-cache instance to reduce our CPU utilisation on redis-cache. We had planned this for weeks in advance due to a capacity planning warning, and we were able to roll this out the day after we had a production page about CPU saturation for redis-cache.

If we hadn't had the capacity planning step in there, we may have noticed this problem much later, and had to scramble to implement mitigations in a high-pressure environment. Instead, we just accelerated our existing timeline slightly, and resolved it with a clean solution.

Timeline

1. 2022-09-21: our capacity planning process creates redis-cache / redisprimarycpu potential saturation to warn about potential future saturation. At this time we noted that it was a risk, but not guaranteed to saturate in the next three months.
1. 2022-10-20: we create Introduce Redis Cluster for redis-ratelimiting (leading to Horizontally Scale redis-cache using Redis Cluster), representing the long-term solution to this.
1. 2022-12-05: having analyzed the workload for Redis Cache, we decide to work on Functional Partitioning for Repository Cache to mitigate Redis saturation forecasts. This is because Introduce Redis Cluster for redis-ratelimiting will take longer to deliver, and we might not be able to deliver it before the forecast saturation date.
1. 2022-12-06: we create Migrate repository cache from redis-cache to new shard on VMs to capture the core work in moving data from one Redis instance to another.
1. 2022-12-14: the new instance is available for testing in our pre-production environment.
1. 2023-01-11: we confirm that the current capacity forecast does not predict saturation, but we expect this is due to the end-of-year holidays.
1. 2023-01-27: we roll out the new redis-repository-cache on the pre and gstg environments in gitlab-com/gl-infra/scalability2050 and gitlab-com/gl-infra/scalability2052.
1. 2023-01-31: we have a production incident warning of imminent redis-cache CPU saturation in gitlab-com/gl-infra/production8335.
1. 2023-01-31: we bring in the work to roll out to gprd as a result, from gitlab-com/gl-infra/production8309. This was intended to happen the following week due to staff availability.
1. 2023-02-10: we have gone from CPU utilisation above 90% to below 70%, and the capacity planning issue (redis-cache / redisprimarycpu potential saturation) issue automatically closes once the data updates for this partitioning.

monitoring / gomemory potential saturation

gitlab-com/gl-infra/capacity-planning42

The Tamland report showed that this component would likely saturate within the next 30 days.
The engineer reviewing the issue saw that the trend lines indicated a problem.
The engineer contacted the Engineering Manager for the team responsible for this component.
The responsible team worked to correct the problem.
When the team was satisfied with their changes, we confirmed that this component was no longer showing in the report.
We also confirmed through source metrics that this component was no longer likely to saturate.

redis-cache / redismemory potential saturation

gitlab-com/gl-infra/capacity-planning45

The Tamland report showed that this component might saturate in the next few months.
The engineer reviewing the issue determined that this saturation point was an artificial limit. It is expected for this component to hover around its maximum without causing problems.
The team worked to exclude these components from the Tamland process.
The issue was resolved.

redis-cache / nodeschedstatwaiting potential saturation

gitlab-com/gl-infra/capacity-planning144

The Tamland report showed potential saturation.
The engineer reviewing this problem could see that outliers in the data (due to an incident) impacted the forecast.
The engineer silenced the alerts and explained why this issue could be closed.

git / kubepoolmaxnodes potential saturation

gitlab-com/gl-infra/capacity-planning31 and gitlab-com/gl-infra/capacity-planning108

The Tamland report showed potential saturation.
The responsible team was contacted and they made changes to address the problem.
They believed they had done enough to prevent the saturation from occurring so they closed the issue.
When Tamland produced its next report, the item was still included.
The responsible team picked up the issue again and found that the metrics reporting the problem had been broken with the previous change.
The team confirmed that the saturation problem was definitely fixed and corrected the metrics to reflect the change.
This example shows that Tamland will continue to notify us of a capacity issue until the metrics show that it is resolved.

SECTION: engineering/infrastructure/cost-management/_index.md

---

Quick Links

Infrafin Board

Cost Management Epic

Dashboards

Mission

Provide Recommendations and Analysis to support and enable GitLab to be as efficient as possible with our cloud spend.

Vision

The idea behind cloud cost management is to work in a cross-functional effort between many different departments including engineering, finance, and data to bring forth the best recommendations possible and prioritize these against all the other customer facing initiatives that are in development.

Strategy

Our strategy follows a generic cost efficiency or data based framework where we have 5 stages

of adoption related to cloud cost efficiency before we get to a stable point. Today we are somewhere between stages 1-4 depending on the problem area.

Stage 1: Basic Cost Visibility

We maintain a list of vendors/coupa renewal dates in this spreadsheet

Stage 2: Cost Allocation

Stage 3: Optimize Usage Efficiency

Stage 4: Measure Business Outcomes vs Spend

Stage 5: Predict Future Spend & Problem Areas

Cost Management Handbook

- Infrafin Board Docs
- GCP CUD Process
- Group Cost Metrics
- How to Engage
- Infra Analyst Board
- Infra Analyst Role
- Learning Resources
  - GCP
  - AWS

Contact Us

Slack

- infrafin is the primary channel for all of GitLab communication in topics that are discussing hosting costs
- eng-data-kpi is the primary channel for high priority requests or questions related to engineering KPIs or RPIs
- gengineeringanalytics is primary channel to ask any general questions to engineering analytics team

---
SECTION: engineering/infrastructure/cost-management/gcp-cud.md

---

What are Committed Use Discounts in GCP?

Google Committed Use Discounts is a way to reduce your compute rate by committing to a set amount of servers of a certain type and in a certain region for a period of time (1 year or 3 year). You will pay this cost whether you use the servers or not, but at a significantly discounted rate compared to the on-demand server rate. This is the equivalent of reserved

instances or compute savings plan in AWS.

- CUD Description
- CUD Restrictions

CUD Dimensions

We can share CUD across projects, but the dimensions CUD are split across are listed below.

- Region
- Machine Type
- Cost Type (CPU vs RAM)

CUD Tracking

- Overview of our current CUD can now be viewed in Sisense

- In Infra, we have a PI to cover 80% or more of our total eligible compute usage with CUD

GCP CUD Purchase Approval Process

1. Fill out CUD Analysis Template with relevant details for new request

CUD Analysis Template (internal)

CUD Analysis should assume the other commitments do not end. CUD renewals should be looked at in a separate analysis so there is no confusion of CUD that cover new and existing covered usage.

2. Fill out New GCP CUD issue template in Finance

Include and ping any engineering manager who will be significantly impacted by the change so they can confirm they do not expect major changes in their usage for the term of the commit. Include the spreadsheet from step 1 in the issue.

The template should include the commitment details, important high level financial details, and the engineering details about which services are most affected by the commitment.

Before the commitment is considered, the infrastructure analyst should talk with the teams that use the majority of the usage that is being committed to make sure there aren't any major changes expected during the term of the commitment.
Those teams should be cc'ed in the issue and if they have any concerns voice them at that time.

Example Issue: https://gitlab.com/gitlab-com/Finance-Division/finance/-/issues/4010

3. Once approved, execute purchase

The infra analyst should make sure there are enough Committed CPU Quota to meet the request and execute this purchase in Billing-Tools GCP Project.

4. Add the commitment to CUD commitments

Add to CUD commitments spreadsheet (internal)

5. Reservation Configuration

Reservations for specific node type are configured in our config-mgmt Terraform repository. This must be updated to ensure efficient utilization of our CUD's, along with the instance choice being used in our infrastructure.

6. Follow-Up

If a team is planning on making a major change to their infrastructure that would affect the commit during the term, they should check with the infrastructure analyst to assess the impact first.

SECTION: engineering/infrastructure/cost-management/group-cost-metrics.md

---

Overview

Group Cost Metrics SSOT

How to Engage

We maintain a set of base level cost metrics for groups to use in Periscope in this Dashboard. These are created in conjunction with a strict definition that explains what is and is not included in the metric, so understanding the definition is crucial to understanding the metric itself.

Currently we aim to have no more than 5 of these metrics per group, but may expand this. Generally we have 1 metric for total cost, 1 metric for total cost MoM growth, and up to 3 additional metrics around specific sub-categories of cost within the group. This is to try and keep the number of metrics manageable as we expand to more groups.

Using the Metrics

The reason for keeping all these type of metrics in one place is so that we have this as the Single Source of Truth (SSOT) for the group level metrics we create and we have a place where everyone can read the exact definitions of the metrics.

When you would like to use these metrics to create a final KPI, for example taking gitaly cost and dividing by total storage amount, you can copy/paste the query used into a subquery in your own dashboard, combine it with the SQL for the rest of the metric and have your final metric. Please make sure to include the original definition as part of your chart description when you do this. If you need help with any part of this, please ask for help in infrafin or gengineeringanalytics slack channel.

Requirements / Limitations

- Include the original metric definition in your newly created charts, or otherwise link back to the source
- Less than 6 base level cost metrics per group

Requesting a New Group Level Cost Metric

If you are a PM or just see a need for a new metric to be added, please read the instructions under "I would like to see a new group or service level cost metric" in How to Engage

---
SECTION: engineering/infrastructure/cost-management/how-to-engage.md

Infrafin Board

infrafin slack channel

Overview

This page will go over preferred ways to engage with infrastructure analyst for various scenarios. For misc questions refer to the buttons above, so you can ask in infrafin slack channel, or open an issue on the Infrafin Board. Slack questions will likely get faster responses, and issues will be triaged in priority according to the Board Criteria. For issues that do not meet the board criteria, please open an issue in Engineering Metrics Board and these will be worked on as availability permits.

For specific asks check the FAQ below to see if your question is answered there.

FAQ

I am a PM and I would like to understand the cost of my service better

This is great to hear! Please start by first doing some initial work on your own by taking to engineers working on your product to understand what are the different infrastructure elements that are used in your product. This is incredibly important, as the infra analyst can only do so much if they can't learn all the details about the service from talking to you. Even better, if there are new things you learn from this, try to get these things documented in the handbook so that we can refer back to those details later.

Once you have done that, reach out in infrafin or schedule a 1hr intro call directly with an infrastructure analyst. We may not use the whole hour, but we want as much time as possible in the first meeting to go over all the various aspects of the product and start narrowing down how we want to go about understanding the cost. From this meeting we will come up with an action plan for you to get the insights you need.

I would like to see a new group or service level cost metric

Open an issue on the engineering metrics board or ask in infrafin slack channel and mention an

infrastructure analyst. They will work with you to get the requirements for the metric. If the data is already available the work will mostly just require collaboration around solidfying the metric definition. If the data does not exist, a longer process will be required to first make creating the metric possible, such as potentially labelling the infrastructure or somehow enabling better tracking in the product to see the metric.

SECTION: engineering/infrastructure/cost-management/infra-analyst-board.md

---

Workflow Board

Criteria

Issues that land on this board should be initiatives that specifically need ad hoc work done by Infrastructure Analyst and do not fall under the criteria to be part of an infrafin initiative.

Labeling

Use label Eng Metrics::Infrastructure Department to designate infra analyst specific issue on this board. This will be done by the team automatically if it exists in the backlog and falls under infra analyst role.

Prioritization

- All issues on this board will automatically de-prioritized from active items on the Infrafin Board
  - If this is an issue please reach out in infrafin slack channel
- Issues on this board will be completed on a best-efforts basis, weights can be applied on a 1-10 scale with 10 being the highest priority
- Total Open Issue Weight that is in progress cannot exceed 10
  - After there is 10 total open and in progress weight on this board, any new issue priorities should be re-evaluated or new issue is moved to backlog
- At 10 given weight if feasible, attempt will be made to resolve within the week, with 1 additional week needed for each 1 point of less weighting

---
SECTION: engineering/infrastructure/cost-management/infrafin-board.md

---

Criteria

While we would like each team to be as efficient as possible, it is not scalable or realistic for us to talk to each individual about one server or a few servers they are spinning up. For that reason we provide learning resources for those individuals to try and price out their own resources and then we define the following minimum requirements for something to be considered

as part of the infrafin board. If the issue meets any of the following criteria and it is within our scope of control (part of central billing that we have visibility into) it can be added as a potential infrafin issue.

- Impact of issue (current or near future) is $10K/quarter or more
- Issue is part of an OKR for the current quarter
- If cost is less than $10K/quarter, but month over month cost has doubled for a particular team or service for an unknown or unplanned reason

Prioritization

Weighting of infrafin issues is done in a similar fashion to RICE framework, but with the 4 factors below considered as part of the weighting. The weights are summed and then divided by 4 to normalize all the weights between 0 and 10, with 10 being the most important or impactful

Weight Factors

Cost Savings

Measure of savings as a result of resolving the issue on a quarterly basis. This measure should always be calculated pre-discounts for weighting to maintain consistent weighting across GCP projects and across vendors.

Factor Value	Weight
< $10K/QTR	0.5
$10K-$25K/QTR	2
$25K-$50K/QTR	4
$50K-$100K/QTR	6
$100K-$200K/QTR	8
$200K-$500K/QTR	9
> $500K/QTR	10

Customer Impact

Factor meant to capture potential negative consequences of a change if production changes to the application are required. There are some cases where an infrafin issue would have a beneficial impact to customers also, but since this is only meant to capture negative effects, a large but beneficial customer impact should be weighted as low customer impact.

Factor Value	Weight
< 5% of Users	10
5-25% of Users	6
25%-50% of Users	3
50-80% of Users	1.5
80% or more of Users	0.5

Future Potential Cost Impact

Measure of future cost that will be incurred if nothing is done to resolve an issue. This measure should always be calculated pre-discounts for weighting to maintain consistent weighting across GCP projects and across vendors. This measure is usually quite speculative, but it is meant to capture both growth in costs or in cases of abuse or un-needed servers the future cost of not addressing these problems.

Factor Value	Weight
< $10K/QTR	0.5
$10K-$25K/QTR	2
$25K-$50K/QTR	4
$50K-$100K/QTR	6
$100K-$200K/QTR	8
$200K-$500K/QTR	9
> $500K/QTR	10

Effort Required

This factor captures the general timeline for how much time and effort is required to resolve the issue.

Factor Value	Weight
< 1 week	10
1 wk - 1mo	7
1 mo - 3mo	5
3 mo - 6 mo	3
6 mo - 1 yr	2
1 yr - 2 yr	1
> 2 yr	0

Infrafin Board

Short Term Saving Initiatives Process

Some savings initiatives can happen on a much shorter time horizon, either because of the urgency or because we can handle the implementation solely from within the infrastructure department. Some examples of this include purchasing CUD from GCP, or AWS savings plan to optimize the cost of our servers across the whole company, or investigating and fixing specific billing anomalies that we see over time.

1. Initial Data Exploration

In this scenario, the issue is already known, potentially because of earlier data exploration or other work, but there is still usually some data exploration to do to understand these issues better.

2. Initial Cost Impact Analysis

Quantify what the impact of the issue at hand is

3. Identify and Partner with SME

Partner with whoever the SME for the area is to find out how the issue can be resolved. If the SME is unknown, the infra analyst will work with the Infra PM to find who the right person is.

4. Resolve the issue

Due to the nature of being bucketed as a short term initiative, these should be relatively low effort, high impact initiatives and generally these should be completed as soon as possible. The weights on these issue will automatically be set to 7

5. Follow-up

After the change has been made, confirm the numbers match close to what was expected.

Longer Term Saving Initiatives Process

These are initiatives that require cross-team collaboration and major changes in a service or infrastructure to happen. Some examples include implementing online registry garbage collection, optimizing our internal CI usage, and changing machine types or storage types for one of our major services.

1. Initial Data Exploration

If the problem area is not well understood, the first step to discovering one of these initiatives is to take a macro view of our infrastructure to understand where we are spending most of our money today, how fast is it growing, and where are we likely to spend more money in the future.
This can be done using whatever tools are available, but today this is mainly done by going into the billing consoles for the various vendors to get a better understanding of where our spend is going.

In the case that an initiative is already well understood you can skip steps 1-2 and move directly to step 3.

2. Decide on focus area

There are many different factors to choose from when it comes to deciding on which issues to tackle, but the main factors which align with our weighting system in descending order of importance would be:

- Cost
- Future Potential Cost
- Customer impact
- Effort Required

There are some ways we can condense this data down to make it easier to compare, such as by using a weighted MoM growth metric around certain dimensions to see what areas of our spend are both very high and also continuing to grow at a rapid pace.

3. Initial Cost Estimate

Once an area to focus on has been chosen, then we can come up with a proposal about how we could change or implement something to optimize our cost, and come up with a cost estimate of what this might be. This will be the quarterly savings label, and this is the main step as an Infra Analyst to do, which will help give finance and PM's a good understanding of how they want to prioritize the issue.

The Infra Analyst should work with the Subject Matter Expert to make sure the assumptions are well defined and that the estimate makes sense given the assumptions.

4. Estimate of effort and customer impact by SME

SME for the service/area being analyzed will decide how difficult and time intensive this solution would be to implement and how many customers it could impact. The SME can partner with the infra analyst to come up with these estimates if needed.

If a SME cannot be identified the Infra Analyst can work with the infra pm to find who the right person is.

5. Prioritization Decision Made

Infra PM takes the issue and decides where it should fit in within other team's roadmap or milestones depending on the priority in context of infrafin items.

If the issue weight is over 7 (and not a short term initiative), infra PM should also consider getting an exec sponsor for the project.

6. Follow-up

Once completed, analyze the real impact of the change and expectations going forward.

Infrafin Initiatives & FP&A

Some infrafin initiatives may be included in our financial plan. Examples of this can be seen in the savings tab of our Vendor Rolling Forecast Spreadsheet. The criteria for initiatives included as part of this are very strict as these would be expected to be completed on time with relatively accurate savings numbers. Labels readyforplan can be used by infra analyst to designate when an initiative has sign off and is ready to be included in the next forecast, and inplan can be used to denote when this has actually been included as part of the forecast.

Criteria to be included in forecast

- firm due date set with sign off from both infra PM and relevant stage PM if necessary
- confidence in savings number > 50%
- readyforplan label on issue with estimated actual savings, meaning that the number takes into account any relevant discounts as well as other initiatives already included in the forecast which may have overlapping savings.

Labeling

- infrafin

used to put issues on the infrafin board

- Savings-Estimate

Estimate of quarterly savings of the initiative. Savings labels should be before any discounts are applied and disregarding other initiatives that amy affect the number. This way, we have a consistent and isolated savings estimate label for each initiative to prioritze by

- readyforplan

used once the issue has passed the criteria mentioned above and is ready to be reviewed and put into the forecast

- inplan

used once the initiative is included into the forecast

SECTION: engineering/infrastructure/cost-management/infrastructure-analyst-role.md

---

Responsibilities

- Optimize infra spend through committed spend programs and supporting enterprise contract deals
- Ad Hoc Analysis of Infrastructure hosting spend
- Dashboard and chart creation to improve observability into infrastructure costs
- Teaching and partnering with PM's and others on how to understand the cost and usage of their services

Working with an infrastructure analyst

There is no magic when reducing or analyzing infrastructure spend, it just requires a good understanding of what our own usage and architecture looks like and how we get billed for it.

Therefore, if requesting help or a review from an infrastructure analyst you should try to provide as much information as possible. Some of the most important pieces of information to provide are:

- description or link to architecture of affected services
- data Source of usage info and what metrics best relate to the cost if known
- Affected product and/or sku of service provider

If one or more of these is not known, then you should partner with the infrastructure analyst so you can define these together.

Infra Analyst Board

The infrastructure analyst role is now part of the centralized engineering analytics team, and serves as a stable counterpart for Infrastructure Department. To open a request for infra analyst please open an issue under engineering metrics board

Learning Track

This section will go through the general skills that are needed to succeed as an infrastructure analyst. The skillset requires a mix of understanding that dips into three main areas: engineering, finance, and data.

- Understanding of major cloud vendors, how they bill, and how to analyze the bill.
  - Most cloud vendors will provide a granular billing report of your company's usage, so this is an important first step to understanding what the company is spending money on
  - Ability to quickly do ad-hoc analysis in any of the cloud vendor billing consoles
- Basic understanding of engineering concepts
  - An infrastructure analyst should know enough about how the applications your company uses to ask the right questions. This includes an understanding of general topics that affect resource utilization like multi-threading.
  - Can identify resource bottlenecks, areas of potential waste or excess usage, etc.
- Basic understanding of finance concepts
  - An infrastructure analyst needs to be comfortable with how finance departments work to be able to integrate with financial processes.
  - Familiar with forecast and cost allocation processes
- Understanding of data infrastructure
  - An infrastructure analyst should be able to work within whatever their company's current data architecture is, and enhance this with the unique data sources they have access to. This requires a greater level of maturity in the company and as a first step just using the cloud vendor billing consoles should suffice.
  - SQL expertise is baseline expectation, some experience with python/R as extra is preferred

---
SECTION: engineering/infrastructure/cost-management/cloud-finops/index.md

Cloud FinOps at GitLab

Overview of the data pipeline

The GCP billing Profit & Loss (P&L) allocation pipeline takes raw GCP billing data and allocates costs to P&L categories for analysis and reporting. It provides a daily overview of GCP costs by P&L category (free. internal, paid).

The pipeline consists of the following key steps:

1. Map raw GCP billing data to infrastructure labels (rptgcpbillinginframappingday)
2. Calculate usage and costs for various infrastructure components like CI runners, storage, etc. and map to P&L categories (models ending in pldaily)
3. Combine all P&L mappings into a unified mapping table (combinedplmapping)

4. Apply the combined P&L mappings to the raw GCP data (rptgcpbillingpldaycombined)
5. Apply the lookback mappings to the raw GCP data (rptgcpbillingplday)
6. Output final allocated costs by P&L category and applies a hierarchy for exploration on Tableau (rptgcpbillingpldayext)

The workflow follows a data transformation pipeline pattern:

- Raw input · Enrichment models · Unified mappings · Allocation · P&L output

Lineage

Combined mappings

The combined P&L mappings consolidate all the individual mapping logic into a single model for simplicity.

Mapping	Source	Metric Used	Scope
buildartifactspldaily	GitLab API	Build artifacts usage per namespace in gigabyte per day	Storage for build artifacts
cirunnerspldaily	GitLab API	CI consumption in ci.minutes per runner type	CI/CD runner usage
containerregistrypldaily	GitLab API	Container registry usage per namespace in gigabyte per day	Storage for container registries
folderpl	GCP hierarchy	Folder path	GCP projects by parent folder
haproxybackendpl	HAproxy metrics from Grafana	Network usage per backend in gigabyte per day	Load balancer egress
haproxybackendratiodaily	HAproxy metrics	Percentage of network usage per backend	Splits load balancer costs by backend
infralabelpl	Config	Infrastructure labels	GCP resources by infrastructure label
namespacepldaily	GitLab API	Namespace plan data	Namespace allocation
projectspl	Config	GCP project ID	Specific GCP project costs
repostoragepldaily	GitLab API	Repo storage usage per namespace in gigabyte per day	Storage for Git repositories

rptgcpbillinginframappingday

Mission, objective, inputs, granularity

- Mission: Map GCP billing data to infrastructure labels.
- Objective: Provide daily GCP billing data with additional metadata for better reporting and analysis.
- Granularity: Daily
- Inputs: Raw GCP billing data

Schema

text

day: date - Date of the record
gcpprojectid: varchar - GCP project identifier
gcpservicedescription: varchar - GCP service description
gcpskudescription: varchar - GCP SKU description
infralabel: varchar - Infrastructure label
envlabel: varchar - Environment label
runnerlabel: varchar - Runner label
usageunit: varchar - Unit of usage
pricingunit: varchar - Unit of pricing
usageamount: float - Amount of usage
usageamountinpricingunits: float - Usage amount in pricing units
costbeforecredits: float - Cost before credits applied
netcost: float - Net cost after credits applied
usagestandardunit: varchar - Standard unit of usage
usageamountinstandardunit: float - Usage amount in standard units

Links

- Model

rptgcpbillingplday

Mission, objective, inputs, granularity

- Mission: Calculate daily GCP billing data by Profit & Loss categories.
- Objective: Provide a daily overview of GCP costs by plcategory for reporting and cost analysis.
- Granularity: Daily
- Inputs: rptgcpbillingpldaycombined, combinedplmapping

Schema

text
dateday: date - Date of the record
gcpprojectid: varchar - GCP project identifier
gcpservicedescription: varchar - GCP service description
gcpskudescription: varchar - GCP SKU description
infralabel: varchar - Infrastructure label
envlabel: varchar - Environment label
runnerlabel: varchar - Runner label
plcategory: varchar - Profit & Loss category
usageunit: varchar - Unit of usage
pricingunit: varchar - Unit of pricing
usageamount: float - Amount of usage
usageamountinpricingunits: float - Usage amount in pricing units
costbeforecredits: float - Cost before credits applied
netcost: float - Net cost after credits applied
usagestandardunit: varchar - Standard unit of usage
usageamountinstandardunit: float - Usage amount in standard units

frommapping: varchar - Source of mapping

Links

- Model

rptgcpbillingpldayext

Mission, objective, inputs, granularity

- Mission: Calculate daily GCP billing data by Profit & Loss categories.
- Objective: Provide a daily overview of GCP costs by plcategory for reporting and cost analysis. This table will be the main source for the cockpit dashboard (Sisense) and the Tableau dashboard
- Granularity: Daily
- Inputs: rptgcpbillingpldaycombined, combinedplmapping

Schema

text
dateday: date - Date of the record
gcpprojectid: varchar - GCP project identifier
gcpservicedescription: varchar - GCP service description
gcpskudescription: varchar - GCP SKU description
infralabel: varchar - Infrastructure label
envlabel: varchar - Environment label
runnerlabel: varchar - Runner label
plcategory: varchar - Profit & Loss category
usageunit: varchar - Unit of usage
pricingunit: varchar - Unit of pricing
usageamount: float - Amount of usage
usageamountinpricingunits: float - Usage amount in pricing units
costbeforecredits: float - Cost before credits applied
netcost: float - Net cost after credits applied
usagestandardunit: varchar - Standard unit of usage
usageamountinstandardunit: float - Usage amount in standard units
frommapping: varchar - Source of mapping

Links

- Model

rptgcpbillingpldaycombined

Mission, objective, inputs, granularity

- Mission: Applies the combined mappings to the billing data
- Objective: Provide a daily overview of GCP costs by plcategory for reporting and cost

analysis.
- Granularity: Daily
- Inputs: rptgcpbillinginframappingday, combinedplmapping

Schema

text
dateday: date - Date of the record
gcpprojectid: varchar - GCP project identifier
gcpservicedescription: varchar - GCP service description
gcpskudescription: varchar - GCP SKU description
infralabel: varchar - Infrastructure label
envlabel: varchar - Environment label
runnerlabel: varchar - Runner label
plcategory: varchar - Profit & Loss category
usageunit: varchar - Unit of usage
pricingunit: varchar - Unit of pricing
usageamount: float - Amount of usage
usageamountinpricingunits: float - Usage amount in pricing units
costbeforecredits: float - Cost before credits applied
netcost: float - Net cost after credits applied
usagestandardunit: varchar - Standard unit of usage
usageamountinstandardunit: float - Usage amount in standard units
frommapping: varchar - Source of mapping

Links

- Model

buildartifactspldaily

Mission, objective, inputs, granularity

- Mission: Map daily build artifacts usage to Profit & Loss categories.
- Objective: Provide daily build artifacts usage data by plcategory for cost analysis and reporting.
- Granularity: Daily
- Inputs: GitLab Storage per namespace statistics, namespacepldaily
- Accuracy rating: Very High
- Completeness rating: Very High

Schema

text
snapshotday: date - Date of the snapshot
financepl: varchar - Profit & Loss category
buildartifactsgb: float - Build artifacts size in GB
percentbuildartifactssize: float - Percentage of build artifacts size

Links

- Model

Examples

sql
```
SELECT snapshotday, financepl, SUM(buildartifactsgb) as totalgb
FROM prod.workspaceengineering.buildartifactspldaily
GROUP BY snapshotday, financepl
ORDER BY dateday desc;
```

Description: This query aggregates build artifacts usage data by P&L category for each snapshot day. It provides a sum of build artifacts size in gigabytes (GB), grouped by the P&L category, giving an insight into the storage requirements and cost allocation for different P&L categories.

cirunnerspldaily

Mission, objective, inputs, granularity

- Mission: Map daily CI runner usage to Profit & Loss categories.
- Objective: Provide daily CI runner usage data by plcategory for cost analysis and reporting.
- Granularity: Daily
- Inputs: GitLab API: ciminutes consumption per type of customer and plan over time
- Accuracy rating: Very High
- Completeness rating: Very High

Schema

text
```
reportingday: date - Date of the report
mapping: varchar - Mapping data
pl: varchar - Profit & Loss category
totalciminutes: number - Total CI minutes used
pctciminutes: number - Percentage of CI minutes used
```

Links

- Model

Examples

sql
```
SELECT reportingday, pl, SUM(totalciminutes) as totalminutes
FROM prod.workspaceengineering.cirunnerspldaily
GROUP BY reportingday, pl
```

```
    ORDER BY reportingday DESC;
```

Description: This query summarizes the total CI minutes used by each P&L category on a given day. It helps in understanding the distribution and usage of CI runners across different P&L categories.

combinedplmapping

Mission, objective, inputs, granularity

- Mission: Combine all Profit & Loss mappings into a single model.
- Objective: Create a unified model to simplify the mapping process and improve maintainability.
- Granularity: Daily
- Inputs: Various PL mappings

Schema

text
```
    dateday: timestampntz - Date of the record
    gcpprojectid: varchar - GCP project identifier
    gcpservicedescription: varchar - GCP service description
    gcpskudescription: varchar - GCP SKU description
    infralabel: varchar - Infrastructure label
    envlabel: varchar - Environment label
    runnerlabel: varchar - Runner label
    plcategory: varchar - Profit & Loss category
    plpercent: float - Percentage of Profit & Loss category
    frommapping: varchar - Source of mapping
```

Links

- Model

Examples

sql
```
    SELECT dateday, frommapping, plcategory, AVG(plpercent) as averageplpercent
    FROM prod.workspaceengineering.combinedplmapping
    GROUP BY dateday, frommapping, plcategory
    ORDER BY dateday DESC;
```

Description: This query provides an average percentage allocation of P&L categories for each mapping by day. It showcases how costs are distributed among different P&L categories based on infrastructure labels.

containerregistrypldaily

Mission, objective, inputs, granularity

- Mission: Map daily container registry usage to Profit & Loss categories.
- Objective: Provide daily container registry usage data by plcategory for cost analysis and reporting.
- Granularity: Daily
- Inputs: GitLab API: Container registry usage per namespace per day
- Accuracy rating: Very High
- Completeness rating: Very High

Schema

text
```
snapshotday: date - Date of the snapshot
financepl: varchar - Profit & Loss category
containerregistrygb: float - Container registry size in GB
percentcontainerregistrysize: float - Percentage of container registry size
```

Links

- Model

Examples

sql
```
SELECT snapshotday, financepl, AVG(containerregistrygb) as averagegb
FROM prod.workspaceengineering.containerregistrypldaily
GROUP BY snapshotday, financepl
ORDER BY snapshotday DESC, financepl DESC;
```

Description: This query provides an average percentage allocation of P&L categories for each infrastructure label by day. It showcases how costs are distributed among different P&L categories based on infrastructure labels.

haproxybackendpl

Mission, objective, inputs, granularity

- Mission: Maps each HAproxy backend to a specific P&L split
- Objective: Enable better allocation and reporting of infrastructure costs by plcategory.
- Granularity: N/A (mapping)
- Inputs: gcpbillinghaproxyplmapping (csv seed)
- Accuracy rating: Medium
- Completeness rating: High

Schema

text

METRICBACKEND: VARCHAR
TYPE: VARCHAR
ALLOCATION: FLOAT

Links

- Model

Examples

sql
SELECT metricbackend, TYPE, AVG(ALLOCATION) as averageallocation
FROM prod.workspaceengineering.haproxybackendpl
GROUP BY metricbackend, TYPE;

Description: This query averages the allocation percentages for each HAproxy backend type. It provides insight into how network usage is distributed across different backend types for P&L categorization.

haproxybackendratiodaily

Mission, objective, inputs, granularity

- Mission: Splits Networking costs into its different backends (SSH, HTTPs, ...)
- Objective: Enable better allocation and reporting of infrastructure costs by plcategory.
- Granularity: N/A (mapping)
- Inputs: HAproxy data also visible on Grafana
- Accuracy rating: Medium
- Completeness rating: High

Schema

text
dateday: timestampntz - Date of the record
backendcategory: varchar - Backend category identifier
usageingib: float - Usage in Gib

Links

- Model

Examples

sql
SELECT dateday, backendcategory, AVG(usageingib) as averageusageratio
FROM prod.workspaceengineering.haproxybackendratiodaily
GROUP BY dateday, backendcategory;

Description: This query calculates the average daily usage ratio for each backend category. It's useful for analyzing the distribution of network costs among different backends on a daily basis.

infralabelpl

Mission, objective, inputs, granularity

- Mission: Map infrastructure labels to Profit & Loss categories.
- Objective: Enable better allocation and reporting of infrastructure costs by plcategory.
- Granularity: N/A (mapping)
- Inputs: gcpbillinginfraplmapping (csv seed)
- Accuracy rating: Medium
- Completeness rating: High

Schema

text
```
infralabel: varchar - Infrastructure label
type: varchar - Type of allocation
allocation: float - Allocation value
```

Links

- Model

Examples

sql
```
SELECT infralabel, type, AVG(allocation) as averageallocation
FROM prod.workspaceengineering.infralabelpl
GROUP BY infralabel, type;
```

Description: This query provides an average allocation value for each infrastructure label and type. It helps in understanding how different infrastructure components are categorized into P&L categories.

namespacepldaily

Mission, objective, inputs, granularity

- Mission: Maintain a daily history of active namespaces and their associated Profit & Loss categories.
- Objective: Provide an historical view of namespace usage by plcategory for analysis and reporting.
- Inputs: GitLab Product Information
- Granularity: Daily

- Inputs: N/A

Schema

text
    dateday: date - Date of the record
    dimnamespaceid: number - Namespace identifier
    dimplanid: number - Plan identifier
    financepl: varchar - Profit & Loss category

Links

- Model

Examples

sql
    SELECT dateday, financepl, COUNT(dimnamespaceid) as totalnamespaces
    FROM prod.workspaceengineering.namespaceplddaily
    GROUP BY dateday, financepl;

Description: This query counts the number of namespaces per P&L category on a given day. It's useful for tracking the usage and distribution of namespaces across different P&L categories.

projectspl

Mission, objective, inputs, granularity

- Mission: Map specific GCP projects to Profit & Loss categories.
- Objective: Provide accurate allocation and reporting of project costs by plcategory
- Granularity: N/A (mapping)
- Inputs: gcpbillingprojectplmapping (csv seed)
- Accuracy rating: High
- Completeness rating: High

Schema

text
    projectid: varchar - Project identifier
    type: varchar - Type of allocation
    allocation: number - Allocation value

Links

- Model

Examples

sql
```
SELECT dateday, financepl, COUNT(dimnamespaceid) as totalnamespaces
FROM prod.workspaceengineering.namespacepldaily
GROUP BY dateday, financepl;
```

Description: This query counts the number of namespaces per P&L category on a given day. It's useful for tracking the usage and distribution of namespaces across different P&L categories.

repostoragepldaily

Mission, objective, inputs, granularity

- Mission: Map daily repository storage usage to Profit & Loss categories.
- Objective: Provide daily repository storage usage data by plcategory for cost analysis and reporting.
- Granularity: Daily
- Inputs: GitLab API: repository storage usage per day and per namespace plan type
- Accuracy rating: Very High
- Completeness rating: Very High

Schema

text
```
snapshotday: date - Date of the snapshot
financepl: varchar - Profit & Loss category
reposizegb: float - Repository size in GB
percentreposizegb: float - Percentage of repository size
```

Links

- Model

Examples

sql
```
SELECT snapshotday, financepl, SUM(reposizegb) as totalgb
FROM prod.workspaceengineering.repostoragepldaily
GROUP BY snapshotday, financepl;
```

Description: This query aggregates the total repository storage usage in GB by P&L category for each day. It helps in understanding the storage requirements and cost allocation for repository storage across different P&L categories.

folderpl

Mission, objective, inputs, granularity

- Mission: Map projects to specific Profit & Loss categories thanks to their path in the organization folder hierarchy.
- Objective: Provide accurate allocation and reporting of sandbox project costs by plcategory.
- Granularity: N/A (mapping)
- Inputs: gcpbillingsandboxprojects (csv seed)
- Accuracy rating: Very High
- Completeness rating: Very High

Schema

text
folder: varchar - GCP project identifier
classification: varchar - Classification category

Links

- Model

rptgcpbillingskusday

Mission, objective, inputs, granularity

- Mission: Map specific SKUs or Service-SKU combinations to Profit & Loss categories.
- Objective: Enable accurate allocation and reporting of specific costs by plcategory.
- Granularity: N/A (mapping)
- Inputs: gcpbillingsingleskuplmapping (csv seed)
- Accuracy rating: Very High
- Completeness rating: Very High

Schema

text
servicedescription: varchar - Service description
skudescription: varchar - SKU description
type: varchar - Type of allocation
allocation: number - Allocation value

Links

- Model

Lookback mappings

The lookback mappings are used to retroactively apply updated profit and loss (P&L) allocations to historical data. This ensures consistency in P&L reporting over time.
In Google's Cloud Billing data, our commitment costs are still incurred on the projects using the eligible compute resources. Once we apply the P&L split to a certain area of costs, the CUD lines are not mapped and must be mapped by looking back on the same perimeters.

There are two lookback mapping models:

- flexcudlookback: applies the P&L split retroactively to the Flex CUD SKUs in the same perimeter
- t2dcudlookback: applies the P&L split retroactively to the T2D CUD SKUs in the same perimeter

Links

- Flex CUD Lookback
- Flex CUD Lookback

Dashboards

(Internal only)Cloud Billing dashboards

Glossary

- P&L: Profit & Loss
  - P&L categories: Profit & Loss labels: either equal to free, internal, or paid.
  - free users: GitLab.com users having a free plan
  - paid users: GitLab.com users having a paid plan
  - internal users: GitLab.com internal users
- GCP (Google Cloud Platform): A suite of cloud computing services provided by Google that runs on the same infrastructure that Google uses internally for its end-user products.
- Pipeline: In this context, a series of data processing steps or stages through which billing data is transformed and analyzed.
- CI Runners: Continuous Integration runners are virtual machines or containers that execute the code in a CI/CD process.
- BigQuery: A fully-managed, serverless data warehouse that enables scalable analysis over petabytes of data. It is a part of the Google Cloud Platform.
- HAproxy: High Availability proxy is software that provides high availability, load balancing, and proxying for applications.
- Tableau: A visual analytics platform transforming the way we use data to solve problems.
- Grafana: An open-source platform for monitoring and observability. It allows you to query, visualize, alert on, and understand your metrics.
- SKU (Stock Keeping Unit): A specific type of item for sale, such as a product or service, and all attributes associated with the item type that distinguish it from other item types.
- CUD (Committed Use Discount): A discount applied in exchange for a commitment to use a minimum level of resources for a specified term with Google Cloud services.
- Flex CUD: A type of Committed Use Discount in Google Cloud Platform that provides flexibility in usage.
- T2D: Refers to a specific type of Google Cloud Platform resource or service, likely in the context of a Committed Use Discount.
- Infrastructure Labels: Tags or labels used to categorize various GCP resources for management and billing purposes.
- Namespace: In GitLab, a namespace provides one place to organize your related projects. Projects in one namespace are separate from projects in other namespaces, which means you can use the same name for projects in different namespaces.
- GitLab API: The programming interface provided by GitLab for automating actions or querying data from GitLab instances.

SECTION: engineering/infrastructure/cost-management/learning/_index.md

---

Vendor Specific Learning

GCP
AWS

Cloud Cost Buckets

The first thing to understand about infrastructure cost is that you can generally categorize it into a few main areas of focus. The areas are listed below in descending order of how much the bucket contributes to overall infra cost in general for most company's. Cloud Provider services may provide abstractions of these buckets, for instance serverless computing, but at their core each service can still be categorized into one of these 3 buckets of infra spend.

- Compute
  - server running cost, includes cpu/ram cost of the hardware used to run these services.
- Storage
  - cost of storing data, generally refers to Object Storage in cloud, but can refer aso refer to physical disk storage, database storage, etc.
- Networking
  - cost of sending data back and forth between customers or services

General Cloud Cost Optimizations

There are some basic cloud optimizations that apply to all resources, regardless of what service is running. Further optimizations or taking action on these recommendations may require more detailed understanding of the application or service running.

Compute Optimizations

- turn off un-needed resources
- downsize over-provisioned resources
- rightsize servers to make most efficient use of their performance characteristics (cpu v ram v storage)
- prefer using newer instance types
  - generally newer instance types will have better cost/performance

Storage Optimizations

- use storage lifecycle policies to reduce cost by archiving un-accessed but needed data
- delete unnecessary data

Networking Optimizations

- Take the most direct network path

- traffic sent between providers/continents/regions is the most expensive, avoid this when possible
- Avoid network hairpinning

Cost Dimensions

There are generic cost dimensions that apply to all company's, as well as some specific dimensions that are similar to other company's, but we define below.

Generic Cost Dimensions

Vendor

- GCP
- AWS
- Azure
- Elastic
- Cloudflare

Service

This could also be called a "product", the two are interchangeable, and this can refer to internal or external services.

- Compute Engine / EC2
- Object Storage / S3
- GitLab CI

Sku

- N1 Predefine Instance Core running in Americas
- Standard Storage US Multi-region

Usage Type

- Compute
- Storage
- Networking

GitLab Specific Dimensions

Feature

- CI Windows Runners
- Gitaly Dedicated Nodes
- Review Apps
- SAST

Tier

- Free
- GitLab Internal
- SaaS - Premium
- SaaS - Ultimate

Resource Links

- MultiCloud Service Comparison

SECTION: engineering/infrastructure/cost-management/learning/aws.md

---

Resource Links

- Cost Optimization
- AWS Cost and Usage Report (CUR)
- AWS Compute Pricing Comparison
- AWS Pricing Calculator
- Instance Rightsizing Recommendations

SECTION: engineering/infrastructure/cost-management/learning/gcp.md

---

Resource Links

- Cost Optimization Principles
- Best practices for optimizing cloud costs
- Billing Export Example Queries
- GCP VM Instance Pricing
- GCP Pricing Calculator

SECTION: engineering/infrastructure/database/_index.md

Database Reliability at GitLab

The group of Database Reliability Engineers (DBREs) are on the Reliability Engineering teams that run GitLab.com. We care most about database reliability aspects of the infrastructure and GitLab as a product.

We strive to approach database reliability from a data driven perspective as much as we can. As such, we start by defining Service Level Objectives below and document what service levels we currently aim

to maintain for GitLab.com.

Database SLOs

We use Service Level Objects (SLOs) to reason about the performance and reliability aspects of the database. We think of SLOs as "commitments by the architects and operators that guide the design and operations of the system to meet those commitments."[^1]

Backup and Recovery

In backup and recovery, there are two SLOs:

SLO	Current level	Definition
DB-DR-TTR	8 hours	Maximum time to recovery from a full database backup in case of disaster
DB-DR-RETENTION-MULTIREGIONAL	7 days	The number of days we keep backups for recovery purposes in Multi-regional Storage class in GCS.
DB-DR-RETENTION-COLDLINE	From 8 to 90 days	The number of days we keep backups for recovery purposes in Coldline storage class in GCS.

The backup strategy is to take a daily snapshot of the full database (basebackup) and store this in Google Cloud Storage. Additionally, we capture the write-ahead log data in GCS to be able to perform point-in-time recovery (PITR) using one of the basebackups. Read more on Disaster Recovery

For DB-DR-TTR we need to consider worst-case scenarios with the latest backup being 24 hours old. Hence recovery time includes the time it takes to perform PITR to recover from archive to a certain point in time (right before the disaster).

We are able to recover to any point in time within the last DB-DR-RETENTION days.

High Availability

For GitLab.com we maintain availability above 99.95%. For the PostgreSQL database, we define the following SLOs:

SLO	Level	Definition
DB-HA-UPTIME	99.9%	General database availability
DB-HA-PERF	p99 < 200ms	99th percentile of database queries runtime below this level.
DB-HA-LOSS	60s	Maximum accepted data loss in face of a primary failure

A DB-HA-UPTIME of 99.9% allows for roughly 45 minutes of downtime per month. Uptime means, the database cluster is available to serve
queries from the application while maintaining other database SLOs.

We allocate a downtime budget of 45 minutes per month for planned downtimes,

although we strive to keep downtime as low as possible. The downtime budget can be used to introduce change to the system. If the budget is used up (planned or unplanned), we stop introducing change and focus on availability (similar to SRE error budgets).

As for DB-HA-PERF, 99% of queries should finish below 200ms.

With DB-HA-LOSS we require an upper bound on replication lag. A write on the primary is considered at risk as long as it has not been replicated to a secondary (or to the PITR archive).

Common Links

To make it easier to find your way around you can find a list of useful or important links below.

Monitoring & Performance Related Tools

As a database specialist the following tools can be very helpful:

- Postgres Checkup:Detailed report about the status of the PostgreSQL database.
- Private Grafana: for both application and system level performance data.
- Performance Bar: type pb in GitLab and a bar with performance metrics will show up at the top of the page. This tool is especially useful for viewing the queries executed and their timings.
- Sherlock: a tool similar to the performance bar but meant for development environments. Sherlock is able to show backtraces and the output of EXPLAIN ANALYZE for executed queries. Enable by starting Rails with env ENABLESHERLOCK=1 bundle exec rails s.
- <https://explain.depesz.com/> for visualizing the output of EXPLAIN ANALYZE.

Dashboards

The following (private) Grafana dashboard are important / useful for database specialists:

- PostgreSQL Database Overview
- Patroni PostgreSQL HA cluster Overview
- PgBouncer Database Proxy Overview

Documentation

Basically everything under <https://docs.gitlab.com/ee/development/databases>, but the following guides in particular are important:

- What requires downtime?
- Adding database indexes
- Post Deployment Migrations
- Background Migrations
- SQL Migration Style Guide
- SQL Query Guidelines
- Infrastructure runbooks and documentation

For various other development related guides refer to
<https://docs.gitlab.com/ee/development/>.

[^1]: From "Database Reliability Engineering", O'Reilly Media, 2017

SECTION: engineering/infrastructure/database/disaster-recovery.md

Moved to Disaster Recovery Policies for GitLab Backups

SECTION: engineering/infrastructure/engineering-productivity/_index.md

> ·· Note: This page is deprecated. The team has been restructured as Development Analytics and Developer Tooling under the Developer Experience Stage.

Tools and processes that previously belonged to Engineering Productivity are being re-assigned or deprecated. The handbook and GitLab docs will be updated to reflect the new changes.

SECTION: engineering/infrastructure/engineering-productivity/gdk.md

Last reviewed: 2021-01-16

- GDK Project
- Issue List
- Epic List

- Please comment, thumbs-up (or down!), and contribute to the linked issues and epics on this category page. Sharing your feedback directly on GitLab.com is the best way to contribute to our vision.
- Please share feedback directly via email, Twitter. There's also a Discord contribute channel you can give us feedback and ask questions in.
- If you're a GDK user, we'd always love to hear from you!

Overview

The GDK team is responsible for the GitLab Development Kit (GDK). The GDK provides a self-contained GitLab environment, which can be run locally or in a virtual machine. It's useful for developers contributing to the GitLab project, or anyone needing to test, experiment with, or validate GitLab functionality.

This category is responsible for improving the usability and reliability of the GDK.
The GDK is essential for locally developing and testing changes.
It is used by nearly all of the product development organization at GitLab as

well as our community of contributors and partners.

It is therefore critical to optimize this experience for two reasons: productivity gains made through tool enhancements have compounding returns, and it makes it easier to get started contributing.

In the longer term, the vision is for the GDK to be a simple, reliable, and flexible tool that allows everyone to contribute. Developers should be able to keep their local development environment up-to-date painlessly, designers should be able to quickly check out a branch and validate a feature they designed, and everyone using the GDK should be running few commands to achieve this, and experience a highly-performant local installation when they do so.

Target audience

The GDK serves as a simple way to set up a local development environment for working on Git